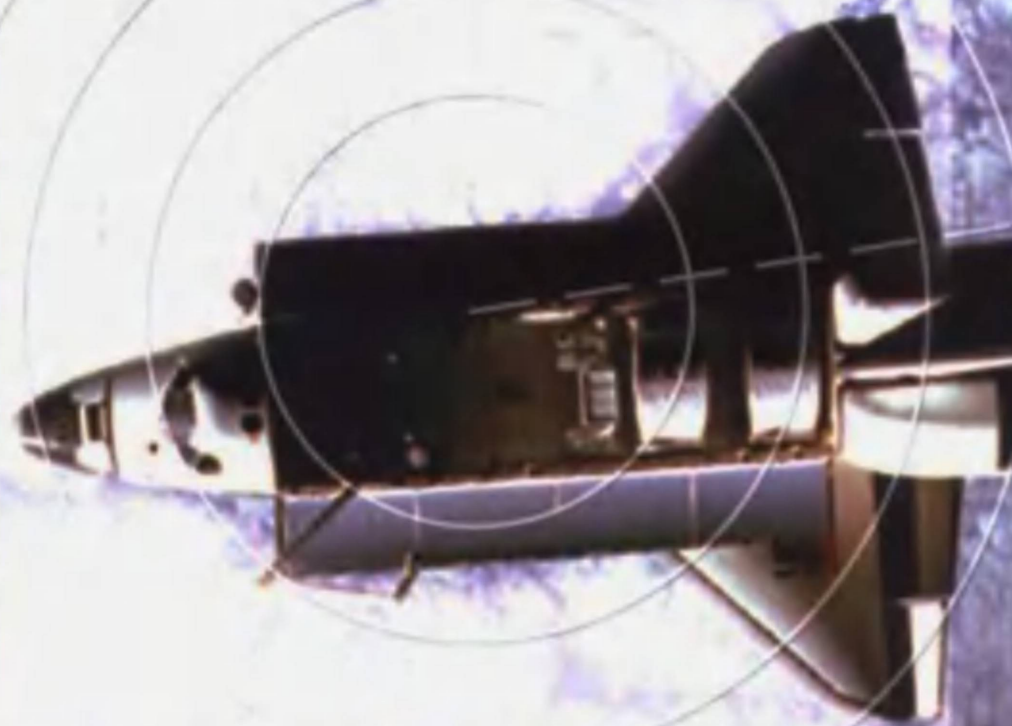


Digital Signal Processing

with Examples in MATLAB



Samuel D. Stearns

Digital Signal Processing with Examples in MATLAB[®]

Samuel D. Stearns



CRC PRESS

Boca Raton London New York Washington DC

Library of Congress Cataloging-in-Publication Data

Stearns, Samuel D.

Digital signal processing with examples in MATLAB/Samuel Stearns.

p. cm. — (Electrical engineering and applied signal processing series)

Includes bibliographical references and index.

ISBN 0-8493-1091-1 (alk. paper)

1. Signal processing—Digital techniques—Computer simulation. 2. MATLAB. I. Title
II. Series.

TK5102 .9.S719 2002

651.382'2—dc21

2002067480

This book contains information obtained from authentic and highly regarded sources. Reprinted material is quoted with permission, and sources are indicated. A wide variety of references are listed. Reasonable efforts have been made to publish reliable data and information, but the authors and the publisher cannot assume responsibility for the validity of all materials or for the consequences of their use.

Neither this book nor any part may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, microfilming, and recording, or by any information storage or retrieval system, without prior permission in writing from the publisher.

The consent of CRC Press LLC does not extend to copying for general distribution, for promotion, for creating new works, or for resale. Specific permission must be obtained in writing from CRC Press LLC for such copying.

Direct all inquiries to CRC Press LLC, 2000 N.W. Corporate Blvd., Boca Raton, Florida 33431.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation, without intent to infringe.

Visit the CRC Press Web site at www.crcpress.com

© 2003 by CRC Press LLC

No claim to original U.S. Government works

International Standard Book Number 0-8493-1091-1

Library of Congress Card Number 2002067480

Printed in the United States of America 3 4 5 6 7 8 9 0

Printed on acid-free paper

**THE ELECTRICAL ENGINEERING
AND APPLIED SIGNAL PROCESSING SERIES**

Edited by Alexander Poularikas

*The Advanced Signal Processing Handbook:
Theory and Implementation for Radar, Sonar,
and Medical Imaging Real-Time Systems*
Stergios Stergiopoulos

The Transform and Data Compression Handbook
K.R. Rao and P.C. Yip

Handbook of Multisensor Data Fusion
David Hall and James Llinas

Handbook of Neural Network Signal Processing
Yu Hen Hu and Jenq-Neng Hwang

Handbook of Antennas in Wireless Communications
Lal Chand Godara

Noise Reduction in Speech Applications
Gillian M. Davis

Signal Processing Noise
Vyacheslav P. Tuzlukov

Digital Signal Processing with Examples in MATLAB®
Samuel Stearns

Forthcoming Titles

Propagation Data Handbook for Wireless Communications
Robert Crane

The Digital Color Imaging Handbook
Guarav Sharma

Applications in Time Frequency Signal Processing
Antonia Papandreou-Suppappola

Smart Antennas
Lal Chand Godara

Pattern Recognition in Speech and Language Processing
Wu Chou and Bing Huang Juang

Nonlinear Signal and Image Processing: Theory, Methods, and Applications
Kenneth Barner and Gonzalo R. Arce

Dedication

*An intelligent man acquires knowledge,
and the ear of the wise seeks knowledge.*

—Proverbs 18:15

Foreword in Memory of Richard W. Hamming (1915-1998)

The information age in which we are living has underlined the importance of signal processing, while the development of solid-state integrated circuits, especially in the form of general-purpose minicomputers, has made practical the many theoretical advantages of digital signal processing. It is for these reasons that a good book on digital signal processing is welcomed by a wide range of people, including engineers, scientists, computer experts, and applied mathematicians.

Although most signal processing is now done digitally, much of the data originates as continuous, analog signals, and often, the result of the digital signal processing is converted back to analog form before it is finally used. Thus, the complex and often difficult to understand relationships between the digital and analog forms of signals need to be examined carefully. These relationships form a recurring theme throughout this book.

The theory of digital signal processing involves a considerable amount of mathematics. The nonmathematically inclined reader should not be put off by the number of formulas and equations in the book, because the author has been careful to motivate and explain the physical basis of what is going on and, at the same time, avoid unnecessarily fancy mathematics and artificial abstractions. It is a pleasure, therefore, to recommend this book to the serious student of digital signal processing. It is carefully written and illustrated by many useful examples and exercises, and the material is selected to cover the relevant topics in this rapidly developing field of knowledge.

This foreword is included in memory of Professor Richard W. Hamming, one of the world's great mathematicians and a pioneer in the development of digital signal processing. The fact that his comments on signal processing, written in 1975 for the progenitor of this text, are just as relevant now as they were when they were written, is a testimony to Dr. Hamming's foresight and genius.

Samuel D. Stearns

Foreword by Delores M. Etter

The combination of digital signal processing techniques with today's computing capabilities has allowed us to address many difficult problems. For example, computers can "understand" human speech, translate it to another language, and then synthesize the translated speech in a new voice. Satellites can transmit information from one part of our world to another in the blink of an eye. Medical doctors can diagnose, and surgically correct, heart valve problems in a baby before it is born. Biometric signals, such as fingerprints and iris scans, can be stored in a "smart card" to allow positive identification of the person using the card.

The capabilities that allow solutions such as the ones mentioned above all use digital signal processing algorithms to extract information from signals collected from the environment around us. These algorithms are based on fundamental principles from mathematics, linear systems, and signal analysis. This text will guide you through the mathematics and electrical engineering theory using real-world applications. It will also use MATLAB, a software tool that allows you to easily implement the signal processing techniques using the computer, and to view the signals graphically. Digital signal processing (DSP) and MATLAB together open up a fascinating new world of possibilities.

The author of this text, Sam Stearns, brings a unique perspective from a career of solving difficult problems at one of our country's finest laboratories — Sandia National Laboratory. He also brings a lifelong passion for education. Sam has been a teacher, a guide, a mentor, and a friend to me for over twenty years. I shall always be in his debt for introducing me to the wonders of digital signal processing.

Dr. Delores M. Etter, Professor, Electrical Engineering, United States Naval Academy at Annapolis, MD, is a former Deputy Under Secretary of Defense for Science and Technology. She is also the author of a number of engineering textbooks, including several on MATLAB.

Preface

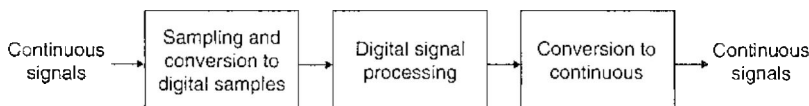
This book is intended to be a text for courses in digital signal processing. It is written primarily for senior or first-year graduate engineering students, and it is designed to teach digital signal processing and how it is applied in engineering.

A progenitor of this text, *Digital Signal Analysis*, was published in 1975. At that time, there were only a handful of texts in the signal processing area, but now there are many texts, and we must give reasons why still another is needed.

One reason is that there is always room for another text in an area like digital signal processing, which is still expanding and finding new applications, provided the text is up to date and covers the fundamental areas, yet does not leave gaps in its development that must somehow be bridged by the reader. We endeavored in this text to provide a complete development of the main subjects in modern digital signal processing. These subjects, we believe, have changed over time in nature and relative importance, and this book is meant to respond to these changes and present the main subjects, or “basics,” applicable in today’s technology.

In a way, it is much more challenging now than it used to be to produce a text on DSP, because the field has grown in so many directions. It is more difficult now in a single book to cover the relevant theory and applications, and yet present the whole subject in a constructive manner, that is, in a manner conducive to teaching that proceeds logically from one topic to the next without omitting derivations and proofs.

A second and perhaps more important reason for this text, in particular, lies in our understanding of current applications of digital signal processing to engineering problems and systems. In most applications, digital signals, which are simply vectors or arrays with finite numbers of discrete elements, are worth processing and analyzing only because they represent discrete samples of continuous phenomena. That is, the engineer who applies the techniques described in a text like this is normally working with at least two, and usually all three, of the following operations:



Our premise is that the reader will find the text more useful, because we do not treat the central operation above as a subject in isolation. We try always to relate digital signal processing to continuous signal processing and to treat digital signals as samples of physical phenomena, just as the

engineer must do when he or she applies digital signal processing to solve real problems.

Other important features of this text include the use of MATLAB® to provide examples of signal processing, digital system design, solutions to exercises, coding and compression algorithms, etc. The MATLAB language has become a standard in signal processing, because it is so easy to understand and use. Its plotting and graphics functions make it ideally suited to signal processing applications. We provide a brief MATLAB tutorial in Chapter 1, so that even if the reader does not use MATLAB, he or she can easily read and understand the algorithms and examples in the text. All of the examples and functions presented in the text, as well as answers to exercises, are included with the software provided for the reader on the CRC PRESS LLC website (www.crcpress.com).

As indicated in the table of contents, other features include areas that are now considered basic but are not always covered in signal processing texts, including an introduction to statistical signal processing, the discrete cosine transform, an introduction to time-frequency and wavelet transforms, coding and compression of signals and other digital data, least-squares system design, and an introduction to adaptive signal processing.

The author of this text is greatly indebted to a number of students, colleagues, and friends, who have been patient and interested enough to comment on large portions of the manuscript. Several have reviewed the entire manuscript and suggested many changes and improvements, in particular, Professors Chaouki Abdallah, Scott Acton, Majid Ahmadi, Victor DeBrunner, Dr. Robert Ives, Jeff Kern, Chip Stearns, and Dr. Li Zhe Tan. The students are those who have attended classes where the manuscript was first used, at the University of New Mexico, and in short courses taught in industry. Other friends and colleagues who have encouraged the author and commented on the text's contents include Dr. Nasir Ahmed, Professors James Matthews, Wasfy Mikhael, Marios Pattichis, Balu Santhanam, Michael Soderstrand, and Dr. Stanley White.

The author is also thankful for his children, who kept asking, "How's the book coming?" (and were really interested in the answer), and to his wonderful wife, Mary, who has patiently read and helped correct the entire manuscript.

Finally, in writing this text, the author has tried to adhere to the descriptions and ideals implied in the two Forewords: First, we hope this project will honor the memory of Richard W. Hamming, one of the world's great mathematicians, who made so many contributions, who inspired so many through his teaching, and who laid so much of the foundation on which signal processing is built today. Second, in this book, we have strived to meet the high standards of clarity and logic in teaching and research set by Professor Delores M. Etter, formerly Deputy Director of Defense Research and Engineering and now Office of Naval Research Distinguished Chair in Science and Technology at the United States Naval Academy.

For MATLAB product information, please contact:

The MathWorks, Inc.

3 Apple Hill Drive

Natick, MA 01760-2098 USA

Phone: 508-647-7000

Fax: 508-647-7001

E-mail: info@mathworks.com

Web: www.mathworks.com

Biography

Samuel D. Stearns has been a Distinguished Member of the Technical Staff at Sandia National Laboratories, Albuquerque, New Mexico, and is now a consultant. His principal technical areas are digital signal processing and adaptive signal processing. His most recent work has been in teaching these subjects at the University of New Mexico and in industrial short courses, as well as in consulting.

Dr. Stearns has taught and advised dissertation research at several universities throughout the United States, including the Universities of Central Florida, Colorado, and New Mexico, and Kansas State and Stanford Universities. He is a Fellow of the IEEE. His Fellow citation reads, "For contributions to education in digital and adaptive signal processing systems and algorithms." He has served in various IEEE activities and has published a number of papers in signal processing, adaptive signal processing, and related areas. He is author or coauthor of the following texts:

Digital Signal Processing with Examples in MATLAB (2003)

Signal Processing Algorithms in MATLAB (1996)

Signal Processing Algorithms in Fortran and C (1993)

Digital Signal Analysis, 2nd Ed. (1990)

Signal Processing Algorithms (1987)

Adaptive Signal Processing (1985)

Digital Signal Analysis (1975)

Contents

1. Introduction	1
1.1 Digital Signal Processing	1
1.2 How to Read this Text	2
1.3 Introduction to MATLAB	2
1.4 Signals, Vectors, and Arrays	3
1.5 Review of Vector and Matrix Algebra	
Using MATLAB Notation	5
1.6 Geometric Series and Other Formulas	12
1.7 MATLAB Functions in DSP	14
1.8 The Chapters Ahead	15
References	16
2. Least Squares, Orthogonality, and the Fourier Series	19
2.1 Introduction	19
2.2 Least Squares	19
2.3 Orthogonality	24
2.4 The Discrete Fourier Series	26
2.5 Exercises	33
References	37
3. Correlation, Fourier Spectra, and the Sampling Theorem	39
3.1 Introduction	39
3.2 Correlation	40
3.3 The Discrete Fourier Transform (DFT)	42
3.4 Redundancy in the DFT	43
3.5 The FFT algorithm	45
3.6 Amplitude and Phase Spectra	48
3.7 The Inverse DFT	51
3.8 Properties of the DFT	52
3.9 Continuous Transforms	58
3.10 The Sampling Theorem	60
3.11 Waveform Reconstruction and Aliasing	62
3.12 Exercises	66
References	69
4. Linear Systems and Transfer Functions	71
4.1 Continuous and Discrete Linear Systems	71
4.2 Properties of Discrete Linear Systems	71

4.3	Discrete Convolution	74
4.4	The z-Transform and Linear Transfer Functions	75
4.5	Poles and Zeros	77
4.6	Transient Response and Stability.....	81
4.7	System Response via the Inverse z-Transform	83
4.8	Cascade, Parallel, and Feedback Structures	84
4.9	Direct Algorithms.....	87
4.10	State-Space Algorithms	89
4.11	Lattice Algorithms and Structures	91
4.12	FFT Algorithms.....	99
4.13	Discrete Linear Systems and Digital Filters	103
4.14	Exercises.....	105
	References	108
5.	FIR Filter Design	111
5.1	Introduction.....	111
5.2	An Ideal Lowpass Filter.....	112
5.3	The Realizable Version	113
5.4	Improving an FIR Filter with Window Functions.....	116
5.5	Highpass, Bandpass, and Bandstop Filters	121
5.6	A Complete FIR Filtering Example.....	125
5.7	Other Types of FIR Filters.....	126
5.8	Exercises.....	130
	References	133
6.	IIR Filter Design	135
6.1	Introduction.....	135
6.2	Linear Phase.....	136
6.3	Butterworth Filters	137
6.4	Chebyshev Filters	141
6.5	Frequency Translations.....	147
6.6	The Bilinear Transformation.....	149
6.7	IIR Digital Filters.....	153
6.8	Other Types of IIR Filters	157
6.9	Exercises.....	162
	References	165
7.	Random Signals and Spectral Estimation.....	167
7.1	Introduction.....	167
7.2	Amplitude Distributions	168
7.3	Uniform, Gaussian, and Other Distributions	171
7.4	Power and Power Density Spectra	177
7.5	Properties of the Power Spectrum	181
7.6	Power Spectral Estimation.....	184
7.7	Data Windows in Spectral Estimation.....	188
7.8	The Cross-Power Spectrum	190

7.9	Algorithms.....	193
7.10	Exercises.....	194
	References.....	197
8.	Least-Squares System Design	199
8.1	Introduction.....	199
8.2	Applications of Least-Squares Design.....	200
8.3	System Design via the Mean-Squared Error	203
8.4	A Design Example.....	207
8.5	Least-Squares Design with Finite Signal Vectors.....	210
8.6	Correlation and Covariance Computation.....	212
8.7	Channel Equalization	215
8.8	System Identification	217
8.9	Interference Canceling.....	220
8.10	Linear Prediction and Recovery	223
8.11	Effects of Independent Broadband Noise	228
8.12	Exercises.....	230
	References.....	238
9.	Adaptive Signal Processing	241
9.1	Introduction.....	241
9.2	The Mean-Squared Error Performance Surface	242
9.3	Searching the Performance Surface.....	244
9.4	Steepest Descent and the LMS Algorithm	249
9.5	LMS Examples	255
9.6	Direct Descent and the RLS Algorithm.....	258
9.7	Measures of Adaptive System Performance.....	263
9.8	Other Adaptive Structures and Algorithms	267
9.9	Exercises.....	268
	References.....	273
10.	Signal Information, Coding, and Compression	277
10.1	Introduction.....	277
10.2	Measuring Information	278
10.3	Two Ways to Compress Signals.....	280
10.4	Entropy Coding.....	282
10.5	Transform Coding and the Discrete Cosine Transform.....	291
10.6	Multirate Signal Decomposition and Subband Coding	298
10.7	Time-Frequency Analysis and Wavelet Transforms.....	308
10.8	Exercises.....	312
	References.....	317
	Index	319

Introduction

1.1 Digital Signal Processing

Digital signal processing (DSP) has become an established area of electrical and computer engineering in relatively recent times. In fact, because all types of signals, when they are processed, are now most often processed in digital form, scientists and engineers in all disciplines have come to at least a nodding acquaintance with the subject in order to understand what their instruments and displays are trying to tell them.

Compared with other areas of electrical engineering that share their analytic tools with DSP, like fields and waves or communication theory or circuits or control theory, DSP has a relatively short history. The author of this text has many colleagues and friends who have built the foundations of DSP from the ground up, and most of them are still active contributors at the time of this writing. The list is too long to print here, but some of them can be found in the lists of “early” references at the end of this chapter¹⁻¹⁵ and subsequent chapters.

This book is the result of an attempt to organize the major contributions of these pioneers and describe subjects that are basic to DSP as it exists currently, that is, subjects that one would find essential to or at least useful in the understanding of most current applications of DSP. There are now many special topics within DSP, such as image processing, digital communications, signal coding and compression, and so on, so that no one is any longer an expert in all areas. But, hopefully, in this text, the reader will find subjects that are basic to (and useful in) all these different areas.

In other words, there is so much diversity in types of signals and types of processing and types of processors that we really only have two choices. One choice is to provide a complete text on a single subject, such as one of those just mentioned or wavelet transforms or digital filters, for example. The second choice is to provide a connected knowledge base consisting of subjects basic to all kinds of digital signal processing, so that the reader may acquire at least the fundamentals but not all of the details.

It is this second option that we have chosen here. We hope that this text will introduce the reader to the basic areas of signal processing and also provide a reference for the reader who wishes to review the fundamentals of DSP.

1.2 How to Read this Text

There are two main ways to read this text. One way is to read it more or less in the order in which it is written and work some of the exercises as you go. Each chapter, especially in the first part of the book, depends on subjects discussed in previous chapters. If the subjects are read and learned in order, the reader should end with a good foundation for work in most areas of signal processing.

The second way is to look in the text for a particular subject, such as filters, coding, spectral estimation, etc. The best way to do this is to look in the index. If your subject is not listed there, look in the table of contents for a related area. If you have at least some familiarity with DSP as well as the basic mathematics needed for analysis, there is no need to read all the text. The treatment of each individual subject is meant to be self-contained, although the text may refer to previous, more basic subjects.

With these two approaches in mind, the reader may now decide whether to read the remainder of this chapter or skip to a subject of interest. The rest of this chapter consists mostly of reviews of some basic mathematics and formulas useful in signal and system analysis, as well as the MATLAB language, which has become a standard in signal processing analysis and system design. If you are familiar with the mathematics and are already a MATLAB user, these reviews may be skipped and treated as reference material.

1.3 Introduction to MATLAB

MATLAB, a product of The MathWorks, Inc., is currently in use by a large fraction of the DSP community. Three principal reasons for this are (1) the syntax allows the user to do most DSP operations with very simple lines of code, (2) with MATLAB's graphics support, one can produce publishable plots with minimal effort, and (3) most importantly, you do not have to be an expert in MATLAB to use MATLAB. The syntax is easy to read and learn and use with almost no prior groundwork.

Most of the DSP examples in this text use the MATLAB language. This does not mean that the reader needs to own or operate MATLAB software, although having access to a computer running MATLAB gives a definite advantage. But the language is useful as a standard for describing DSP operations and algorithms. For readers unfamiliar with MATLAB notation, we begin here with some basics. If you do not own or operate MATLAB, you may wish to view the use of MATLAB in this text as a convenient and easy-to-read standard

system for describing signal processing procedures. For more depth in MATLAB, the reference by Etter¹⁹ at the end of this chapter not only describes MATLAB but also addresses several of the topics in this text, with applications.

MATLAB uses single expressions called *commands*, which may be assembled into sets of commands called *m-files* (because the file extension is “m”) or *functions*, which may be called by other m-files or functions. In this text, when you see lines inside a text box, these will usually be MATLAB expressions. For example,

```
» x=4 ;  
» y=[2 , 3 , 4] ;  
» z=[1 2 3; 4 5 6] ;
```

(1.1)

Each of these lines is an individual expression (command) as indicated by the *command prompt* at the beginning of the line. Each line ends with a semicolon; if not, then MATLAB would echo the results of the line as in (1.2) below. *Row elements* of an array may be separated by commas (as in *y*) or by spaces (as in *z*). *Rows* are separated by semicolons (as in *z*). The results of (1.1) are seen in (1.2) when the MATLAB expression “*x,y,z*” is entered without a semicolon at the end.

```
» x, y, z  
x=  
    4  
y=  
    2    3    4  
z=  
    1    2    3  
    4    5    6
```

(1.2)

From basic expressions like these, we can proceed rapidly and simply in the following sections to expressions that accomplish complicated DSP operations.

1.4 Signals, Vectors, and Arrays

High-level computer languages such as MATLAB are designed to process ordered sequences of elements, that is, *variables*, *vectors*, and *matrices*. As used in this text, these three terms form a hierarchy in the sense that:

- A variable is a single element (integer, real, or complex) [like *x* in (1.1)].
- A vector is an ordered sequence of variables [like *y* in (1.1)].
- A matrix is an ordered sequence of vectors [like *z* in (1.1)].

We usually use *array* as an inclusive term to designate a vector or a matrix. Sometimes our use of *vector* here is confusing at first, because we are used to vectors in electromagnetics or mechanics—in three-dimensional space—with exactly three components. But these are really just ordered sequences of three variables and are thus vectors in the sense used here, and we must now allow more dimensions in order to have vectors that represent signals in a “signal space,” which we define as follows.

Figure 1.1 shows a sampled waveform and its corresponding signal vector. When we say a waveform is *sampled*, we mean that its amplitude is measured and recorded at different times. Usually, these different times are equally spaced in the time domain, but not always. We assume that the samples are equally spaced. The interval between samples is called the *sampling interval* or *time step*. In Figure 1.1; the time step is $\Delta t \equiv T = 1$. In this example, there are ten sample values, and the *signal vector* consists of the ordered sequence of the ten integer samples. Because each sample is a variable, we may say that the *signal space* of the sampled waveform in Figure 1.1 has ten dimensions, one for each sample. Thus, in this sense, the signal space of a sampled waveform has as many dimensions, or *degrees of freedom*, as there are samples in the waveform.

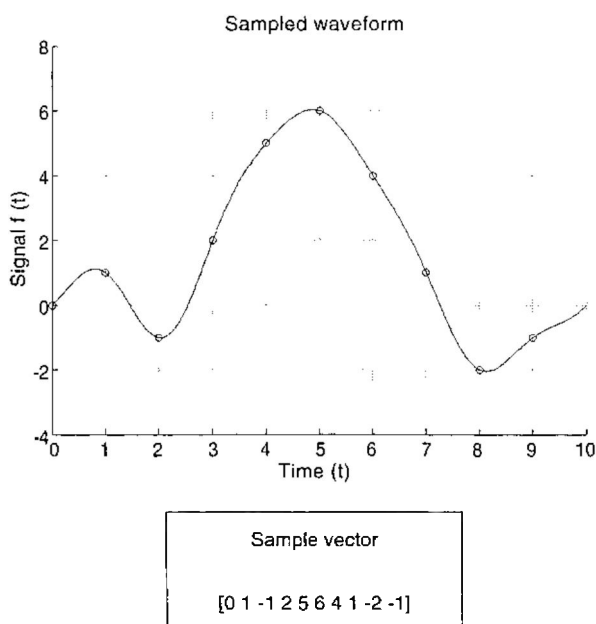


FIGURE 1.1

Sampled waveform and sample vector with time step $T = 1$.

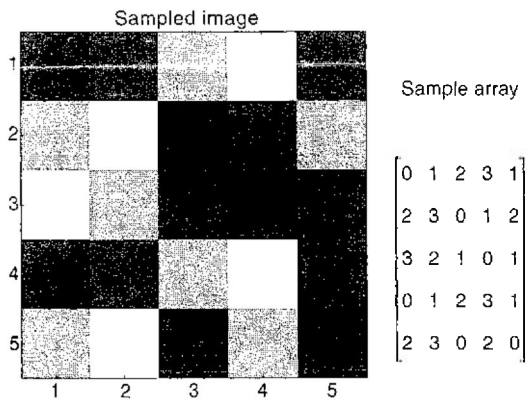


FIGURE 1.2
Sampled gray-scale image and sample array.

Figure 1.2 illustrates a (very small) grey-scale image and its corresponding sample array. Each element of the image is called a *pixel* (picture element), and in this example, an element can take on only one of four values, 0, 1, 2, or 3, and these values cover the range from black to white in the order given; that is, 0 = black, and 3 = white. The digital *image array* is shown next to the image, and in this case, we may say that the signal space associated with the image has 25 dimensions, one for each sample (variable) in the image.

Thus, through processes of sampling and analog-to-digital conversion that depend on the quantities being measured, signals and images in the real, continuous world are converted to vectors and matrices of integers in the discrete world of DSP. It is with these kinds of digital data that the subject of DSP begins. We should note that the examples in Figures 1.1 and 1.2 are simple for the sake of illustration. In real situations, waveform vectors and image arrays have large numbers of elements, as may be seen in some of the examples in this text. Real images, which consist of thousands or millions of pixels, typically have 256 grey levels, not just four as in Figure 1.2, and color images have many more levels to a pixel. Moreover, *video* signals, with the most data of all, consist of sequences of images.

1.5 Review of Vector and Matrix Algebra Using MATLAB Notation

In the rest of this chapter, before we begin discussing DSP fundamentals in Chapter 2, we review some of the basic mathematics required to understand DSP systems, beginning in this section with vector and matrix algebra. We have seen how vectors and matrices are used to represent digitized signals

and images. Vectors and matrices also appear in other ways in DSP, as we will see later. In modern DSP, and especially in languages like MATLAB, vector and matrix sums, products, etc. are the basic processing operations. These operations are mostly just demonstrated here. The development is not as formal as it would be in a text on linear algebra or matrix theory. A summary of all operations is presented in Table 1.1. If the reader is already familiar

TABLE 1.1

Examples of Array Operations

Operation	Symbol	Example	Remarks
Sum or difference	+ or -	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 0 & -1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 4 & 6 \end{bmatrix}$ $[1 \ 2 \ 3] - 2 = [-1 \ 0 \ 1]$	Dimensions must be the same unless one variable is a scalar.
Matrix product	*	$\begin{bmatrix} 1 & 2 & 0 \\ 3 & 4 & 2 \end{bmatrix} * \begin{bmatrix} 0 & -1 \\ 1 & 2 \\ -1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 2 & 7 \end{bmatrix}$ $2 * [1 \ 2 \ 3] = [2 \ 4 \ 6]$	Inner dimensions must be the same unless one variable is a scalar.
Array product	.*	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} .* \begin{bmatrix} 0 & -1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & -2 \\ 3 & 8 \end{bmatrix}$ $2 .* [1 \ 2 \ 3] = [2 \ 4 \ 6]$	Dimensions must be the same unless one variable is a scalar.
Transposes	'	$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}' = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix};$ $[1 + 2j \ 3 - 4j]' = \begin{bmatrix} 1 - 2j \\ 3 + 4j \end{bmatrix}$ $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}.' = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix};$ $[1 + 2j \ 3 - 4j]'. = \begin{bmatrix} 1 + 2j \\ 3 - 4j \end{bmatrix}$	$j = \sqrt{-1}$. Rows become columns. Elements are conjugated with ' but not with .'.
Exponentiation	^	$2.^{[1 \ 2 \ 3]} = [2 \ 4 \ 8]$ $[1 \ 2 \ 3].^2 = [1 \ 4 \ 9]$ $\begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \end{bmatrix}.^{\begin{bmatrix} 0 & 1 & 2 \\ 2 & 1 & 0 \end{bmatrix}} = \begin{bmatrix} 0 & 1 & 4 \\ 9 & 4 & 1 \end{bmatrix}$	Dimensions must be the same unless one variable is a scalar.
Backslash	\	$a * b = c, \text{ where } a = \begin{bmatrix} 5 & 3 \\ 2 & 2 \end{bmatrix} \text{ and } c = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ $b = a \backslash c = \begin{bmatrix} 5 & 3 \\ 2 & 2 \end{bmatrix} \backslash \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$	Solves sets of linear equations.

with the operations, the table alone should suffice as a convenient reference.

To begin, we note that a vector is either an array with one row (*row vector*) or an array with one column (*column vector*). Thus, rules and statements about arrays are generally true for vectors.

The sum or difference of two arrays is obtained simply by adding or subtracting the individual elements. Thus,

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} + \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 5 \\ 7 \\ 9 \end{bmatrix} \quad \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} + \begin{bmatrix} 0 & -1 & 4 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 7 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 0 \\ 3 & 4 & 1 \end{bmatrix} + \begin{bmatrix} -1 & 1 & 2 \\ 2 & -2 & -1 \end{bmatrix} = \begin{bmatrix} 0 & 3 & 2 \\ 5 & 2 & 0 \end{bmatrix} \quad (1.3)$$

The MATLAB expressions for these operations are quite similar. For example,

```

>> [1,2,0; 3,4,1] + [-1,1,2; 2,-2,-1]
ans =
    0     3     2
    5     2     0

```

(1.4)

But, we do not really need the MATLAB versions here, where conventional expressions are more acceptable. Notice that in order for a sum or difference operation to make sense, the two arrays must have the same dimensions, that is, 3×1 , 1×3 , and 2×3 in (1.3). An exception occurs when one of the arrays is a scalar or single variable. In this case, the scalar or variable is added to or subtracted from each element of the second array. Thus,

$$3 + \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 4 & 5 \\ 6 & 7 \end{bmatrix} \quad (1.5)$$

In this text, we use letters without subscripts to represent single vectors and arrays like those in (1.3) and (1.5) and letters with subscripts to represent the elements. For example,

$$a = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} \quad (1.6)$$

The first index of each element designates the row, and the second the column. Indices generally begin at one. If a is a vector, then only one index is required.

Two kinds of array *products* are used most often in DSP. The first is the ordinary matrix product with symbol $*$, formed by summing products of

row elements in the first array times column elements in the second. If a is an array with M rows and N columns, and if b is an array with N rows and K columns, the elements of the array product $c = a * b$ are given by the following:

$$c_{ij} = \sum_{n=1}^N a_{in} b_{nj}; \quad 1 \leq i \leq M \quad \text{and} \quad 1 \leq j \leq K \quad (1.7)$$

For example, suppose a , b , and c are defined as follows:

$$a = \begin{bmatrix} 1 & -1 & 0 \\ 0 & -1 & 1 \\ 2 & 0 & 1 \end{bmatrix}; \quad b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}; \quad c = \begin{bmatrix} -1 \\ 1 \\ 2 \end{bmatrix} \quad (1.8)$$

Then we can see from the matrix product definition in (1.7) that the array equation $a * b = c$ is an expression of three simultaneous equations. That is,

$$\begin{aligned} b_1 - b_2 &= -1 \\ -b_2 + b_3 &= 1 \quad \text{and} \quad a * b = c \\ 2b_1 + b_3 &= 2 \end{aligned} \quad (1.9)$$

are equivalent expressions of three linear equations in three unknowns. Notice how the matrix product $a * b$ is obtained by summing products along rows of a and down columns of b .

In DSP, it is often necessary to solve simultaneous linear equations like (1.9), and we will return to this subject shortly.

The second kind of product, used just as often in DSP as the matrix product defined in (1.7), is obtained simply by multiplying corresponding elements together. To do this, the two arrays must have the same dimensions. This kind of operation is designated with the symbol $.*$ (dot star) to distinguish it from the matrix product ($*$). Thus, if a and b are $M \times N$ arrays, an element of the *array product* (as opposed to matrix product), $c = a .* b$, is given by

$$c_{ij} = a_{ij} b_{ij}; \quad 1 \leq i \leq M \quad \text{and} \quad 1 \leq j \leq N \quad (1.10)$$

To illustrate the array product, suppose we wish to “fade” the image in Figure 1.2 by making the rows lighter from the bottom of the image to the top. Then we could construct a “fading matrix,” Q , and use the array product to produce the faded version of the original sample array (S) in Figure 1.2.

For example, let

$$Q = \begin{bmatrix} 5 & 5 & 5 & 5 & 5 \\ 4 & 4 & 4 & 4 & 4 \\ 3 & 3 & 3 & 3 & 3 \\ 2 & 2 & 2 & 2 & 2 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \tag{1.11}$$

The fading operation is then given by

$$Q.*S = \begin{bmatrix} 5 & 5 & 5 & 5 & 5 \\ 4 & 4 & 4 & 4 & 4 \\ 3 & 3 & 3 & 3 & 3 \\ 2 & 2 & 2 & 2 & 2 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} .* \begin{bmatrix} 0 & 1 & 2 & 3 & 1 \\ 2 & 3 & 0 & 1 & 2 \\ 3 & 2 & 1 & 0 & 1 \\ 0 & 1 & 2 & 3 & 1 \\ 2 & 3 & 0 & 2 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 5 & 10 & 15 & 5 \\ 8 & 12 & 0 & 4 & 8 \\ 9 & 6 & 3 & 0 & 3 \\ 0 & 2 & 4 & 6 & 2 \\ 2 & 3 & 0 & 2 & 0 \end{bmatrix} \tag{1.12}$$

The original and faded versions of the image in Figure 1.2 are shown in Figure 1.3. Note that there are 16 gray levels in the faded version, compared with four in the original.

The array product may be used in the same way to impose an “envelope” on a waveform. For example, the vector f consisting of 1000 samples of a unit sine wave with 50 samples per cycle is produced by the following MATLAB expressions:

```
»n=[1:1000];  
»f=sin(2*pi*n/50);
```

(1.13)

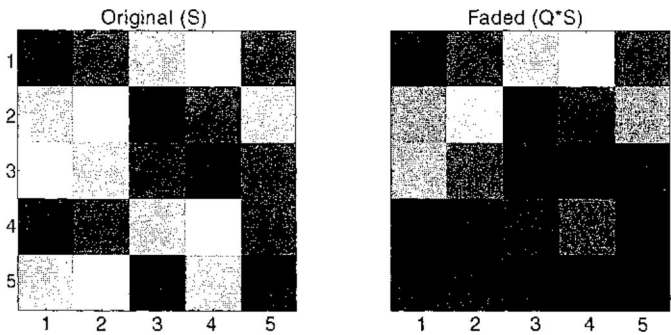
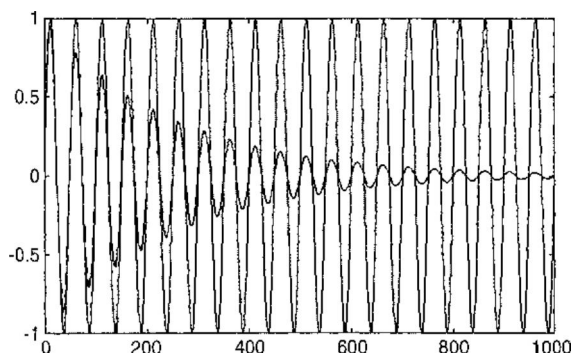


FIGURE 1.3
Original and faded images in equation (1.12).

**FIGURE 1.4**

Plots of undamped and damped sine waves resulting from (1.13) and (1.14).

Notice how the row vector n causes the sine function to produce f , also a row vector with 1000 elements. The sine wave is then “dampened” to produce the vector g by imposing a decaying exponential envelope using the array product $(*)$, and then both f and g are plotted, as follows:

```
»g=exp(-n/250).*f;
»plot(n,f,n,g)
```

(1.14)

In (1.14), each element (f_n) of f is multiplied by the exponential function $e^{-n/250}$ to produce an element (g_n) of g . The results of the MATLAB plot function are shown in Figure 1.4, illustrating the ease of producing such a plot. It is also easy to add grid lines, labels, etc., as we will later see in examples.

Having reviewed array sums and products, we turn to two operations that work with single arrays. First is the *transpose* with symbol $'$ (prime), which changes rows to columns. Thus, each element a_{ij} in array a becomes a_{ji} in a' . The transpose is illustrated in Table 1.1. An important feature of the MATLAB transpose applies when the elements of the array are complex. Then, in a' , the transposed elements are *conjugated*. If the elements are complex and conjugation is not desired, then $'$ (dot prime) is used in place of $'$ (prime). Note the examples in Table 1.1.

The second operation is the matrix *inverse* with symbol $^{-1}$ or *inv*. If a is a square matrix, $\text{inv}(a)$ or a^{-1} is another square matrix such that $a * a^{-1} = I$, where I is the *identity matrix* with ones on the diagonal and zeros elsewhere. The inverse of any 2×2 matrix, which is needed often in DSP, is

$$a^{-1} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}^{-1} = \frac{1}{a_{11}a_{22} - a_{12}a_{21}} \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix}; \quad a * a^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I \quad (1.15)$$

For a matrix to have an inverse, it must be square, because only then will the matrix product $a * a^{-1}$ be valid. The matrix must also be *nonsingular*, no row may be a linear combination of other rows. This, in turn, means that the determinant^{16,17} of the matrix must not vanish. In the case of (1.15), the determinant of a is $a_{11}a_{22} - a_{12}a_{21}$. In this text, we will prefer the *backslash* operation, which is described next after exponentiation, to the inverse.

Array *exponentiation* is also useful in DSP and may be done either with scalars or arrays. In this text, we use only the MATLAB $\wedge$ (dot hat) operation, which is similar to $.*$ (dot star) but produces powers instead of products. Thus,

$$\begin{bmatrix} 3 & 2 & 1 \\ -1 & 0 & 2 \end{bmatrix} \wedge 2 = \begin{bmatrix} 9 & 4 & 1 \\ 1 & 0 & 4 \end{bmatrix} \quad \begin{bmatrix} 3 & 2 & 1 \\ -1 & 0 & 2 \end{bmatrix} \wedge \begin{bmatrix} 2 & -1 & 0 \\ -1 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 9 & 0.5 & 1 \\ -1 & 0 & 4 \end{bmatrix} \quad (1.16)$$

On the left, each element of the first array is squared; and on the right, each element of the first array is raised to the corresponding power in the second array.

The final array operation we review here is the *backslash* ($\backslash$) operation, which is useful in DSP for solving linear equations like those in (1.9). We saw in connection with (1.9) that a set of linear equations may be expressed in the form $a * b = c$. The solution is found by multiplying on the left by a^{-1} , that is, $b = a^{-1} * c$. The *backslash* notation, $b = a \backslash c$, is equivalent, but in MATLAB, the latter is preferred, because it uses methods that produce the smallest roundoff errors. Thus, in summary,

$$\text{If } a * b = c, \text{ then } b = a \backslash c \quad (1.17)$$

For example, the system in (1.9) is solved as follows:

$$b = a \backslash c = \begin{bmatrix} 1 & -1 & 0 \\ 0 & -1 & 1 \\ 2 & 0 & 1 \end{bmatrix} \backslash \begin{bmatrix} -1 \\ 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix} \quad (1.18)$$

The operations described in this section are the main algebraic operations we use in DSP. We will also need relational and logical operations, but they will be introduced in terms of the MATLAB language as they are used. This is the painless way to learn MATLAB—learning necessary operations and functions as they are needed and memorizing those most often used. Table 1.1, consisting of examples of the operations discussed in this section, is provided as a reference.

1.6 Geometric Series and Other Formulas

The *geometric series* is used repeatedly to simplify expressions in DSP. The basic form of the series with N terms is

$$\begin{aligned}\sum_{n=0}^{N-1} x^n &= 1 + x + x^2 + \cdots + x^{N-1} \\ &= \frac{1 - x^N}{1 - x}\end{aligned}\quad (1.19)$$

The reader can prove this result by induction: It is obviously true for $N = 1$. And, you can easily show that if it is true for N , it is true for $N + 1$, and therefore true for all N . Moreover, if the magnitude of x is less than one, the infinite version of (1.19) is as follows:

$$\sum_{n=0}^{\infty} x^n = \frac{1}{1 - x}; \quad |x| < 1 \quad (1.20)$$

In DSP, x in the geometric series is often a complex exponential variable of the form e^{jk} , where $j = \sqrt{-1}$ and k is a real constant. Thus, for example,

$$\sum_{n=0}^{N-1} e^{j\frac{2\pi n}{N}} = \frac{1 - e^{j2\pi}}{1 - e^{j2\pi/N}} = 0 \quad (1.21)$$

Expressions in DSP are often simplified using the geometric series in this way. Furthermore, the *complex plane* is a useful aid to understanding why sums like this vanish. On the complex plane, $e^{j\theta}$ becomes a unit vector at angle θ measured counterclockwise from the positive real axis. For example, all the terms in (1.21) with $N = 8$ are plotted as unit vectors in Figure 1.5.

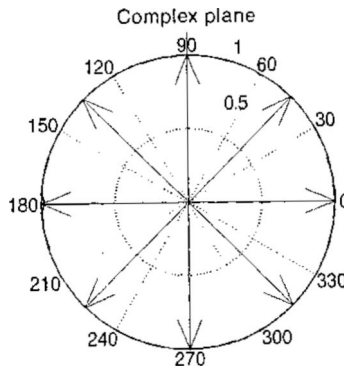


FIGURE 1.5

All the terms in (1.21) with $N = 8$ on the complex plane.

TABLE 1.2

Trigonometric Identities

$\sin \theta = \frac{1}{2j}(e^{j\theta} - e^{-j\theta})$	$\cos \theta = \frac{1}{2}(e^{j\theta} + e^{-j\theta})$
$e^{j\theta} = \cos \theta + j \sin \theta$	
$\sin(\theta + \alpha) = \sin \theta \cos \alpha + \cos \theta \sin \alpha$	$\cos(\theta + \alpha) = \cos \theta \cos \alpha - \sin \theta \sin \alpha$
$\sin \theta \sin \alpha = \frac{1}{2}(\cos(\theta - \alpha) - \cos(\theta + \alpha))$	$\cos \theta \cos \alpha = \frac{1}{2}(\cos(\theta + \alpha) + \cos(\theta - \alpha))$
$\sin \theta \cos \alpha = \frac{1}{2}(\sin(\theta + \alpha) + \sin(\theta - \alpha))$	
$\sin^2 \theta + \cos^2 \theta = 1$	$\cos^2 \theta - \sin^2 \theta = \cos 2\theta$
$\sin^2 \theta = \frac{1}{2}(1 - \cos 2\theta)$	$\cos^2 \theta = \frac{1}{2}(1 + \cos 2\theta)$
$\tan \theta = \frac{\sin \theta}{\cos \theta}$	

That is, the vector at 3 o'clock is the term for $n = 0$, the vector at 12 o'clock is for $n = 2$, etc. We can see from the symmetry of the plot that the sum of all eight vectors is zero as in (1.21).

Some of the most useful *trigonometric identities* are provided in Table 1.2 as a reference for later chapters. Trigonometric functions, especially sine and cosine functions, appear in different combinations in all kinds of harmonic analysis—Fourier series, Fourier transforms, etc. The identities that give sine and cosine functions in terms of exponentials are important, because they allow us to find sums of sines and cosines using the geometric series as in the example above. For example, with the exponential form of the sine or cosine function, we can easily show from (1.21) that when K is an integer,

$$\sum_{n=0}^{KN-1} \sin\left(\frac{2\pi n}{N}\right) = 0 \quad \text{and} \quad \sum_{n=0}^{KN-1} \cos\left(\frac{2\pi n}{N}\right) = 0 \quad (1.22)$$

That is, a sum of equally spaced samples of any sine or cosine function is zero, provided the sum is over an integral number of cycles, or periods, of the function.

In addition to the trigonometric formulas in Table 1.2, some additional mathematical formulas and operations are summarized in Table 1.3. These are used frequently in DSP design and analysis as well as in later chapters of this text.

The last two lines in Table 1.3, in which u and v are functions of an independent variable (say t) are especially useful with continuous transforms and transfer functions, because it is often necessary to differentiate or integrate products or quotients of functions when deriving transforms or transfer functions. For example, suppose we wish to integrate the product $\sin \alpha t e^{j\omega t}$ over

TABLE 1.3

Additional Identities and Operations

if $x = a^y$, then $\log_a x = y$	$\log_{10} x = \log x \log_{10} e$, where $\log = \log_e$
$\sinh x = \frac{1}{2}(e^x - e^{-x})$	$\cosh x = \frac{1}{2}(e^x + e^{-x})$
$\sinh^{-1}(x) = \log(x + \sqrt{x^2 + 1})$	$\cosh^{-1}(x) = \log(x + \sqrt{x^2 - 1})$; $x > 1$
$\frac{d(uv)}{dx} = u \frac{dv}{dx} + v \frac{du}{dx}$	$\frac{d(u/v)}{dx} = \left[v \frac{du}{dx} - u \frac{dv}{dx} \right] / v^2$
$\int u dv = uv - \int v du$	

an indefinite interval of t . If we set $u = \sin \alpha t$ and $dv = e^{j\omega t} dt$, then $v = (1/j\omega)e^{j\omega t}$ and $du = \alpha \cos \alpha t dt$, and the last line in Table 1.3 becomes the following:

$$\int \sin \alpha t e^{j\omega t} dt = \frac{1}{j\omega} \left(\sin \alpha t e^{j\omega t} - \alpha \int \cos \alpha t e^{j\omega t} dt \right) \quad (1.23)$$

At first, this result seems only to make the problem worse, but if we use the same method on the right-hand integral in (1.23), this time with $u = \cos \alpha t$, the result is as follows:

$$\int \cos \alpha t e^{j\omega t} dt = \frac{1}{j\omega} \left(\cos \alpha t e^{j\omega t} + \alpha \int \sin \alpha t e^{j\omega t} dt \right) \quad (1.24)$$

Now the integral on the right in (1.24) is the same as the original integral on the left in (1.23), and we can combine the two equations to give the desired result:

$$\int \sin \alpha t e^{j\omega t} dt = \frac{e^{j\omega t}}{\alpha^2 - \omega^2} (j\omega \sin \alpha t - \alpha \cos \alpha t) \quad (1.25)$$

Thus, applications of the last line in Table 1.3, which is called *integration by parts*, are often useful for integrating products of functions.

1.7 MATLAB Functions in DSP

MATLAB *functions*, as we mentioned in Section 1.3, are m-files that can be called by other m-files. In the MATLAB system, these files are implemented in a way that is especially advantageous in the development of DSP algorithms and systems. As you go through this text, there will be opportunities to write your own functions to use in doing exercises and then (hopefully) to apply in your own occupation. The MATLAB language thus helps people in DSP and other research and development areas to work efficiently by stockpiling low-level operations, rather than having to re-develop them for each new project.

The *row_vec* function is a simple example. Suppose your MATLAB programs often require the conversion of unknown arrays to row vectors. Then, even though the conversion is simple, you could write the following function once and avoid doing the task repeatedly:

```
function v=row_vec(A)
% v=row_vec(A)
%
% Converts any array A into row vector v.
% If A is 1xN, v=A. If A is Nx1, v=A.'.
% If A is a matrix, v=A scanned by rows.
AT=A';
v=AT(:)';
```

(1.26)

MATLAB allows you to store functions like this in your own system and then use them in other functions. If you are like most of us, you may eventually forget how to use the function, hence the comment (%) lines in (1.26). The comments allow one to request help at the MATLAB command prompt as in the following example:

```
> help row_vec
v=row_vec(A)

Converts any array A into row vector v.
If A is 1xN, v=A. If A is Nx1, v=A.'.
If A is a matrix, v=A scanned by rows.
```

(1.27)

That is, the *help* request at the command prompt causes the initial comment lines of the function to be printed. If you are new to MATLAB, it is a good idea to try the *help* command (you can even enter "*help help*") and get instructions for some of the commonly used functions. It is also a good idea, as you read this text and work exercises, to form the habit of developing functions to do tasks that require repeating, and thus build your own personal library. In DSP, there is a real advantage in making a practice of doing this.

Copyright 1996 by John Wiley & Sons, Inc.

1.8 The Chapters Ahead

In the chapters ahead, we develop the fundamentals of DSP more or less from the ground up. As indicated in the Preface, we are attempting to cover the subject from an applied standpoint, assuming that the reader is planning to process digitized samples of real, continuous signals.

Chapter 2 is a review of least-squares analysis, orthogonality, and the Fourier series, all of which are needed for later chapters but may not be necessary for readers familiar with these subjects. Chapters 3 through 4 proceed through Fourier spectra, sampling and measurement of signals, the sampling theorem, linear algorithms, structures, and transfer functions. Chapters 2 through 4 are meant to be read successively, unless the reader is already familiar with the material.

The remaining chapters, from Chapter 5 on, cover different subjects basic to DSP—digital filters, statistical signal processing, least-squares system design, and so on. These chapters depend on Chapters 2–4 but are relatively independent of each other, and therefore not meant to be read in a particular order. The reader may choose to look only at chapters of current interest.

All of the chapters except this chapter have exercises at the end. This chapter is meant primarily for background and reference, but in each of the rest of the chapters, the exercises serve the purpose not only of providing examples of the topics covered in the chapter, but also of introducing topics and applications not covered in the chapter. In this sense, the exercises are an essential component of the text.

References

Digital Signal Processing

1. Shannon, C.E., *The Mathematical Theory of Communication*, University of Illinois Press, Urbana, IL, 1949.
2. Blackman, R.B. and Tukey, J.W., *The Measurement of Power Spectra*, Dover, New York, 1958.
3. Hamming, R.W., *Numerical Methods for Scientists and Engineers*, McGraw-Hill, New York, 1962.
4. Kuo, F.K. and Kaiser, J. F., Eds., *System Analysis by Digital Computer*, John Wiley & Sons, New York, 1967, chap. 7.
5. Gold, B., et al., *Digital Processing of Signals*, McGraw-Hill, New York, 1969.
6. Rabiner, L.R. and Rader, C.M., Eds., *Digital Signal Processing*, IEEE Press, New York, 1972.
7. Oppenheim, A.V. and Schaffer, R.W., *Digital Signal Processing*, Prentice Hall, Englewood Cliffs, NJ, 1975.
8. Rabiner, L.R. and Gold, B., *Theory and Application of Digital Signal Processing*, Prentice Hall, Englewood Cliffs, NJ, 1975.
9. Peled, A. and Liu, B., *Digital Signal Processing: Theory, Design, and Implementation*, John Wiley & Sons, New York, 1976.
10. Tretter, S.A., *Introduction to Discrete-Time Signal Processing*, John Wiley & Sons, New York, 1976.
11. Bellanger, M., *Digital Processing of Signals*, John Wiley & Sons, New York, 1984.
12. Orfanidis, S., *Optimum Signal Processing: An Introduction*, MacMillan, New York, 1985.
13. Ludeman, L.E., *Fundamentals of Digital Signal Processing*, Harper & Row, New York, 1986.

14. Roberts, R.A. and Mullis, C.T., *Digital Signal Processing*, Addison-Wesley, Reading, MA, 1987.
15. Oppenheim, A.V. and Schaffer, R.W., *Discrete-Time Signal Processing*, Prentice Hall, Englewood Cliffs, NJ, 1989, chaps. 4, 5.

Vectors, Matrices, and Linear Algebra

16. Strang, G., *Linear Algebra and Its Applications*, 3rd ed., Harcourt Brace, Orlando, 1988.
17. Lipshutz, S., *Schaum's Outline of Theory and Problems of Linear Algebra*, 2nd ed., Schaum's Outline Series, McGraw-Hill, New York, 1991.
18. Lyons, L., *All You Wanted to Know About Mathematics but Were Afraid to Ask*, vols. 1 and 2, Cambridge University Press, Cambridge, 1998.

MATLAB

19. Etter, D.M., *Engineering Problem Solving with MATLAB*, Prentice Hall, Englewood Cliffs, NJ, 1993.
20. Moler, C., Ed., *The Student Edition of MATLAB*, The MathWorks, Inc. (latest edition).

2

Least Squares, Orthogonality, and the Fourier Series

2.1 Introduction

The three topics reviewed in this chapter have been fundamental to DSP since its beginning. An understanding of these subjects provides insight into almost every area of spectral analysis and DSP system design. The three topics are related to each other. The use of orthogonal functions greatly simplifies the process of finding a *least-squares fit* of a linear function to a set of data. The Fourier series is an important example of a linear least-squares function, and the Fourier series is a series made up of orthogonal functions.

In this chapter, we provide a brief review of these topics as they are applied in DSP. The list of references at the end of the chapter includes more general texts on these subjects.

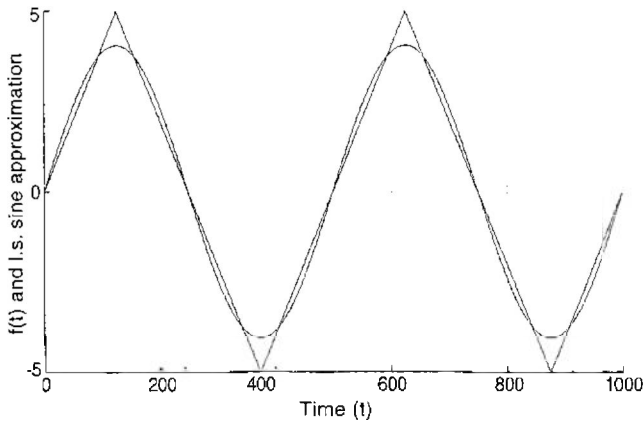
2.2 Least Squares

The principle of *least squares* is used often in DSP with signals and other functions of one or more variables. In this discussion, we will assume functions of just one variable (time), that is, waveforms, because they are simplest, but the concepts apply to images as well.

To begin, suppose we have two continuous functions, $f(t)$ and $\hat{f}(c, t)$, where c is a constant or an array of constants (that is, c is not a function of t). The elements of c may then be selected to make $\hat{f}(c, t)$ a *least-squares approximation* to $f(t)$ with respect to a specified range of t , say from t_1 to t_2 . If c is selected in this manner, then the *total squared error*,

$$\text{TSE} = \int_{t_1}^{t_2} (f(t) - \hat{f}(c, t))^2 dt \quad (2.1)$$

is as small as possible.

**FIGURE 2.1**

$f(t)$ and continuous least-squares approximation.

An example of a continuous least-squares fit is illustrated in Figure 2.1. The first function, $f(t)$, is the triangular wave with period 500, plotted from $t_1 = 0$ to $t_2 = 1000$. In the second function, $\hat{f}(c, t) = c \sin(2\pi t/500)$, c is adjusted to minimize the TSE in (2.1), and so $\hat{f}(t)$ is a least-squares fit to $f(t)$ in the interval $[t_1, t_2] = [0, 1000]$. Note that there is only one constant to adjust in this example, that is, c is a scalar in this case, adjusted to minimize the TSE.

In DSP, least-squares approximations are made more often to *discrete* (sampled) data rather than to continuous data. Instead of starting with a continuous function, $f(t)$, we start with a *sample vector* with N elements, $f = [f_1 \ f_2 \ \dots \ f_N]$. If the approximating function is again $\hat{f}(c, t)$, the total squared error, similar to (2.1), is now given by the following:

$$\text{TSE} = \sum_{n=1}^N (f_n - \hat{f}(c, nT))^2 \quad \checkmark (2.2)$$

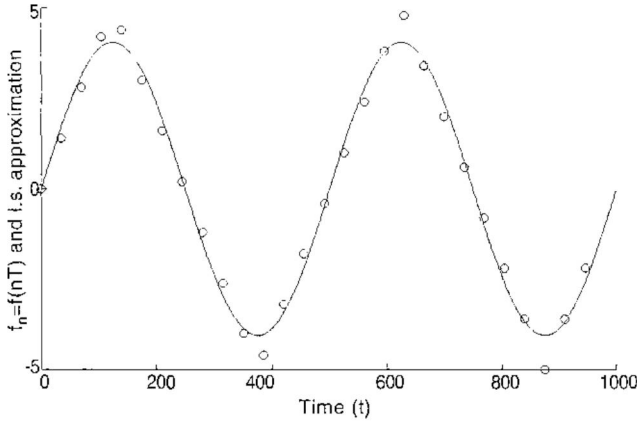
where f_n is the n th element of f , and T is the time step (interval between samples) described in Chapter 1, Section 1.4.

An example of a discrete least-squares fit is illustrated in Figure 2.2. The functions are the same as in Figure 2.1, but here $f(t)$ is sampled, and c is adjusted to minimize the TSE in (2.2). The time step (sampling interval) in Figure 2.2 is $T = 35$, and the data vector, f , has $N = 28$ elements.

A *linear* least-squares approximation occurs when $\hat{f}(c, t)$ is a linear function of the elements of c . A *least-squares polynomial* of the form

$$\hat{f}(c, t) = c_1 + c_2 t + c_3 t^2 + \dots + c_M t^{M-1} \quad \checkmark (2.3)$$

is an example of a linear least-squares function. In this case, there are M functions of the form t^m with m beginning at zero. More generally, suppose $g_m(t)$ is the m th function of t , $g_{mn} \equiv g_m(nT)$ is the value of $g_m(t)$ at $t = nT$, and

**FIGURE 2.2**

Least-squares approximation to sampled data with time step $T = 35$.

there are M functions in all. Then the linear least-squares function, beginning this time with $m = 1$, may be written

$$\hat{f}(c, nT) = c_1 g_{1n} + c_2 g_{2n} + c_3 g_{3n} + \cdots + c_M g_{Mn} \quad (2.4)$$

and the TSE in (2.2) becomes

$$\begin{aligned} \text{TSE} &= \sum_{n=1}^N (f_n - \hat{f}(c, nT))^2 \\ &= \sum_{n=1}^N \left(f_n - \sum_{m=1}^M c_m g_{mn} \right)^2 \end{aligned} \quad (2.5)$$

The TSE, viewed as a function of the vector c , is minimized by setting its gradient vector to zero; that is,

$$\nabla(\text{TSE}) = \left[\frac{\partial \text{TSE}}{\partial c_1} \quad \frac{\partial \text{TSE}}{\partial c_2} \quad \cdots \quad \frac{\partial \text{TSE}}{\partial c_M} \right] = [0 \ 0 \ \cdots \ 0] \quad (2.6)$$

Using (2.5) in (2.6) produces M equations of the following form:

$$\begin{aligned} \frac{\partial \text{TSE}}{\partial c_m} &= -2 \sum_{n=1}^N g_{mn} \left[f_n - \sum_{k=1}^M c_k g_{kn} \right] \\ &= -2 \left[\sum_{n=1}^N g_{mn} f_n - \sum_{n=1}^N \sum_{k=1}^M c_k g_{mn} g_{kn} \right] = 0; m = 1, 2, \dots, M \end{aligned} \quad (2.7)$$

Dividing both sides of (2.7) by -2 and exchanging the summation order in the second term, we obtain

$$\sum_{k=1}^M c_k \sum_{n=1}^N g_{mn} g_{kn} = \sum_{n=1}^N f_n g_{mn}; \quad m = 1, 2, \dots, M \quad \checkmark (2.8)$$

This is the set of linear equations to be solved for the least-squares coefficient vector, c . To simplify the notation, we use the following vector notation for the N samples of each function in (2.8):

$$G_m = [g_{m1} \ g_{m2} \ \dots \ g_{mN}]; \quad f = [f_1 \ f_2 \ \dots \ f_N]; \quad c = [c_1 \ c_2 \ \dots \ c_M]' \quad (2.9) \quad \checkmark$$

Note that c is defined as a *column* vector. Then, using the MATLAB operators described in Chapter 1, Table 1.1, all M equations in (2.8) are given by

$$\begin{bmatrix} G_1 * G'_1 & G_1 * G'_2 & \dots & G_1 * G'_M \\ G_2 * G'_1 & G_2 * G'_2 & \dots & G_2 * G'_M \\ \vdots & & & \vdots \\ G_M * G'_1 & G_M * G'_2 & \dots & G_M * G'_M \end{bmatrix} * \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_M \end{bmatrix} = \begin{bmatrix} f * G'_1 \\ f * G'_2 \\ \vdots \\ f * G'_M \end{bmatrix} \quad \checkmark (2.10)$$

This expression of (2.8) provides an example of the use of the vector product (*) and transpose (') operations in Table 1.1. An element $G_m * G'_k$ of the left matrix is seen to be $\sum_{n=1}^N g_{mn} g_{kn}$ in (2.8), an element $f * G'_m$ of the right vector is seen to be $\sum_{n=1}^N f_n g_{mn}$, and therefore, the m th row of (2.10) is the same as (2.8) for each value of m . Once we see that (2.10) expresses the M equations in (2.8), we can go one step further by defining

$$G = [G'_1 \ G'_2 \ \dots \ G'_M] \quad \checkmark (2.11)$$

so that there is a row of G for each *sample* (n) and a column for each *function* (M). Then, we may express (2.10) and, therefore, (2.8), as follows:

$$G' * G * c = (f * G)' \quad \checkmark (2.12)$$

This result is a general expression of the discrete linear least-squares equations. In accordance with Table 1.1 in Chapter 1, the solution is then given by

$$c = (G' * G) \backslash (f * G)' \quad \checkmark (2.13)$$

The best way to see how these results work is to do an example.

Suppose we have a signal for which the elements of the sample vector $f = [f_1 \ f_2 \ \dots \ f_N]$ with $N = 12$ are the first 12 samples in Figure 2.2 (also shown

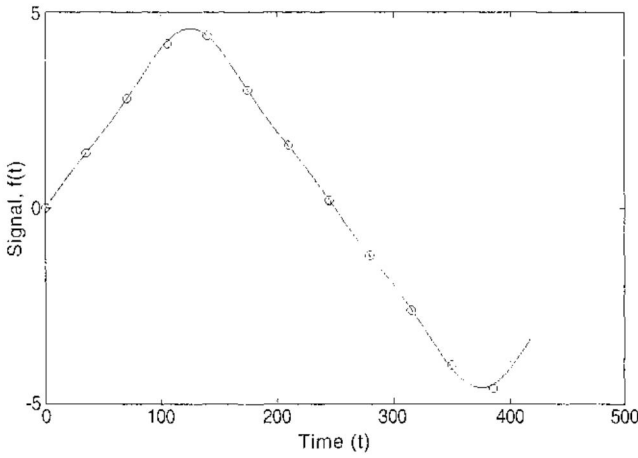


FIGURE 2.3
Least-squares fit plotted by the expression in (2.17).

in Figure 2.3), as follows:

$$f = [0.00 \ 1.40 \ 2.80 \ 4.20 \ 4.40 \ 3.00 \ 1.60 \ 0.20 \ -1.20 \ -2.60 \ -4.00 \ -4.60] \quad (2.14)$$

Instead of the single sine function in Figure 2.2, suppose we now wish to fit the data in f to the sum of three sine functions,

$$\hat{f}(C, nT) = c_1 \sin(2\pi nT/500) + c_2 \sin(6\pi nT/500) + c_3 \sin(10\pi nT/500); \quad n = 0, 1, \dots, 11 \quad (2.15)$$

That is, g_{mn} in (2.5) is $g_{mn} = \sin(2\pi(2m-1)nT/500)$, with n beginning at zero here. The matrix G in (2.11) is now a matrix with $M=3$ columns and $N=12$ rows, with each element given by $g_{mn} = \sin(2\pi(2m-1)nT/500)$. [Note that the indices are switched, because the G_m vectors in (2.11) are transposed.] The following MATLAB commands could be used to generate G :

```
m=[1:3];
n=[0:11];
T=35;
G=sin(2*pi*(n')*(2*m-1)*T/500);
```

(2.16)

The key operation here is the vector product inside the sine argument that causes G to be a 12×3 matrix. Having found G , we now have all the elements needed to express the least-squares equations in (2.12). If you are using MATLAB, the following expressions, along with (2.16) and row vector f , will solve the equations as in (2.13) and plot the data and the continuous approximation as shown in Figure 2.3.

$$\begin{aligned} c &= (G' * G) \setminus (f * G)'; \\ n &= [0:12*35-1]'; \\ \text{arg} &= 2 * \pi * n / 500; \\ \text{fhat} &= [\sin(\text{arg}) \quad \sin(3 * \text{arg}) \quad \sin(5 * \text{arg})] * c; \\ \text{plot}(35 * [0:11], f, 'o', n, \text{fhat}); \text{grid} \end{aligned} \quad (2.17)$$

Note in (2.17) that we have changed the time step from 35 to 1 to produce the continuous plot.

One other point may be made concerning the least-squares equations, (2.8) and (2.12): The equations may not be solvable. As with any set of M linear equations with M variables, a unique solution does not exist unless the equations are independent. This will not be the case if the number of data points, N , is less than M .¹ Hence, $N \geq M$ is always assumed in our discussions of least-squares. Furthermore, if $N = M$, the function and the data have the same degrees of freedom, the fit is exact, and the TSE is zero. We will not discuss solvability further here, but the points just discussed are illustrated in Exercises 1(a) and 2 at the end of this chapter.

In summary, the *discrete least-squares process* is used in many areas of DSP. It consists of fitting a linear combination of continuous functions with adjustable coefficients to a set of data samples in a way such that the *total squared error* (TSE) defined in (2.5) is minimized. This, in turn, involves solving the linear equations in (2.8), or in matrix form in (2.12), for the vector c of *least-squares coefficients*.

In the next section, we will see that when the continuous functions are orthogonal with respect to the set of data samples (we will explain what this means), the process of finding the vector c is greatly simplified.

2.3 Orthogonality

In addition to the principle of least squares, orthogonality is another important concept used often in DSP. The word comes from a Greek word (*orthos*) implying a perpendicular or right-angled relationship. In mathematics, two row vectors a and b are said to be orthogonal if their inner product given by $a * b'$ is equal to zero. When a and b have just two or three elements, then the original meaning of orthogonality holds, and the vectors are perpendicular to

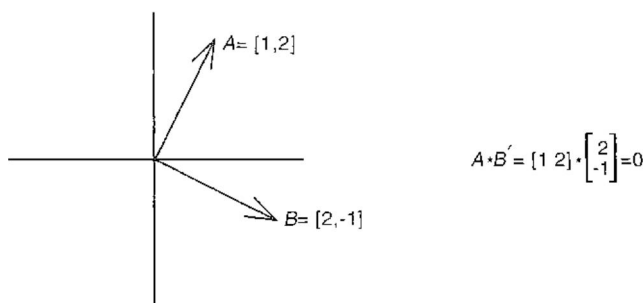


FIGURE 2.4

Orthogonal vectors A and B form a right angle at the origin.

each other, as illustrated in Figure 2.4. When vectors have more than three elements, we lose the spacial picture of orthogonality, but the definition, that is, $a * b' = 0$, holds for any number of elements, provided only that a and b have the same number of elements.

In Chapter 1, Section 1.4, we discussed the concept of *waveform vectors* in *signal space*. We say two such waveform vectors are *orthogonal* when their product vanishes as just described. Sometimes these vectors occur from measurements, that is, analog-to-digital conversion, and sometimes they occur from the evaluation of functions. In the latter case, orthogonality must be evaluated *with respect to a specified set of samples*. That is, the vectors in this case consist of specified samples of the functions. For example, suppose we have N samples of two functions of t , $a(t)$, and $b(t)$, sampled at $t = 0, T, \dots, (N-1)T$ to produce elements $a_0, \dots, a_{N-1}$ and $b_0, \dots, b_{N-1}$. These are then the elements of two vectors, a and b , and the condition for orthogonality with respect to the N samples is $a * b' = 0$; that is,

$$a * b' \equiv \sum_{n=0}^{N-1} a_n b_n = 0 \quad (2.18) \quad \text{✓}$$

Suppose, for example, that $a_n = \sin(2\pi n/N)$ and $b_n = \cos(2\pi n/N)$. Then, using the result in Chapter 1, Equation (1.22), we could say that the two functions are orthogonal with respect to equally spaced samples over an integral number of cycles of the functions. But we could also show that the two functions are, in general, *not* orthogonal with respect to other sets of equally spaced samples.

Knowing when functions are orthogonal is important in the least-squares process, because when they are, the process is simplified. Look again at (2.4), in which the approximating function, $\hat{f}(c, nT)$, consists of a linear combination of M functions, $g_1, g_2, \dots, g_m, \dots, g_M$. As in (2.9), each of these functions has a sample vector given by $G_m = [g_{m1} \ g_{m2} \ \dots \ g_{mN}]$. Now, suppose the M

functions are *mutually orthogonal* with respect to the N samples, that is,

$$G_m * G'_k = 0; \quad m \neq k \quad \checkmark (2.19)$$

Then all the off-diagonal matrix elements in (2.10) are zero, and each of the equations in (2.10) becomes a simple expression of the form

$$G_m * G'_m * c_m = f * G'_m; \quad m = 1, 2, \dots, M \quad \checkmark (2.20)$$

Noting that c_m is a scalar and that the two vector products are also scalars, we see that the solution for c_m in (2.20) is

$$c_m = \sum_{n=1}^N f_n g_{mn} / \sum_{n=1}^N g_{mn}^2; \quad m = 1, 2, \dots, M \quad \checkmark (2.21)$$

Thus, when the least-squares functions are orthogonal with respect to a given set of samples, we do not need to solve simultaneous equations to get the least-squares coefficient vector. The vector elements can be determined one by one as in (2.21).

In the next section, we discuss the discrete Fourier series as an example of least-squares with orthogonal functions.

2.4 The Discrete Fourier Series

A discrete Fourier series consists of combinations of sampled sine and cosine functions like those we have been using as examples in this chapter. The Fourier series is named after J. B. J. Fourier (1768–1830). It forms the basis of a branch of mathematics called *harmonic analysis*, which is applicable to the study of all kinds of natural phenomena, including the motion of stars and planets and atoms, acoustic waves, radio waves, etc. The contributions of Fourier are a principal part of the foundation of DSP. Fourier's basic theorem, put into the DSP terms we are using here, states that any sample vector, no matter what its size or origin, can be exactly reproduced by summing a sufficiently long series of harmonic functions.

We must first describe what we mean by a *harmonic function*. To do this, we must first review a few basic properties of signals. Suppose we begin with a sample vector like the one we used earlier in this chapter, $x = [x_0, x_1, \dots, x_{N-1}]$. (Notice that here the indexes range from 0 to $N - 1$ instead of 1 to N . This is admittedly annoying, but it cannot be helped, because

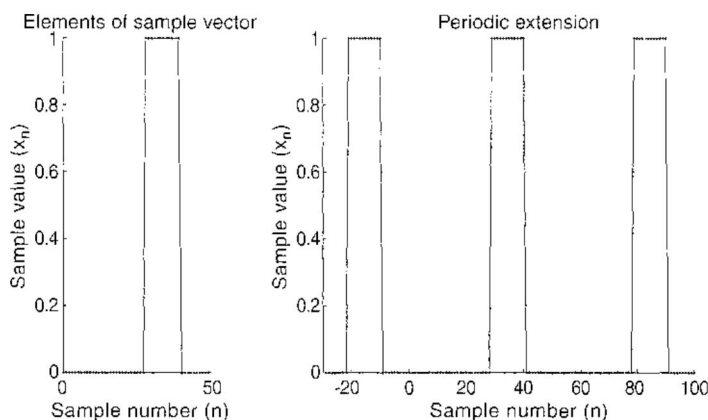


FIGURE 2.5
Sample vector and periodic extension; $N = 50$.

sometimes we want the array index to start at 1, and sometimes we want the time of the first sample to be 0. We often need this convention in using MATLAB for DSP—for example, whenever the first element in an array is the sample at $t = 0$.)

We say the *fundamental period* of x is N samples. When we say this, we are really imagining that the samples of x repeat, over and over again, in the time domain. In other words, we are really thinking of a *periodic extension* of x , illustrated in Figure 2.5. The elements of x are plotted on the left in Figure 2.5. They are shown connected with line segments to show approximately the original continuous function, $x(t)$, before sampling. The periodic extension of x on the right is formed by repeating x every N samples endlessly in both directions as shown.

Any function that repeats like the one on the right in Figure 2.5 is called a *periodic function*. The *fundamental period* is N samples, or NT seconds, where T is the time step in seconds. The *fundamental frequency* is the reciprocal of the fundamental period, or $f_0 = 1/NT$ Hertz (Hz). This, as we can see, is the number of times the function repeats (number of periods) in one second; that is, “Hertz” means “cycles per second.” For example, suppose $T = 1$ ms (0.001 s) in Figure 2.5. Then the fundamental period is $NT = 50$ ms, and the fundamental frequency is $f = 20$ Hz.

We also use two other measures of frequency besides f in this text. One of them is $\omega = 2\pi f$ rad/s (radians per second). There are 2π radians in one cycle (rotation on the complex plane), and the factor 2π converts cycles/s to rad/s. We will introduce the third measure of frequency later.

We now introduce the *discrete Fourier series* as a *least-squares approximation* in the form of (2.4). We use two equivalent forms of the Fourier series in this text. These are given on the first and last lines of the following:

$$\hat{x}_n = \hat{x}(c, nT) = \sum_{m=-(M-1)}^{M-1} c_m e^{jmn\omega_0 T} \quad (2.22)$$

$$\begin{aligned} &= c_0 + \sum_{m=1}^{M-1} c_m e^{jmn\omega_0 T} + c_{-m} e^{-jmn\omega_0 T} \\ &= c_0 + \sum_{m=1}^{M-1} (c_m + c_{-m}) \cos mn\omega_0 T + j(c_m - c_{-m}) \sin mn\omega_0 T \\ &= \frac{a_0}{2} + \sum_{m=1}^{M-1} a_m \cos mn\omega_0 T + b_m \sin mn\omega_0 T \end{aligned} \quad (2.23)$$

In these expressions of the Fourier series, ω_0 is the fundamental frequency of the series in rad/s, $t = nT$ is the continuous time variable, and a_m , b_m , and c_m are coefficients similar to those in (2.4). Notice that for c_m , m ranges from $-(M-1)$ to $M-1$ instead of 0 to $M-1$, and that $b_0 = 0$. The relationships of the coefficients in the two forms of the series are summarized in Table 2.1. They can be derived from (2.23) and the line just above (2.23). The prime (') in Table 2.1 indicates the complex conjugate.

Neglecting the constant terms, a_0 and c_0 , the Fourier series in its second form (2.23) is easily seen to be periodic with period $2\pi/\omega_0$, because each sine and cosine function goes through m cycles during this period.

Furthermore, a Fourier series is a sum of *harmonic functions*. The terms for $m = 1$ in (2.22) and (2.23) are called *fundamental terms*. The terms for $m = 2$ are *second harmonic terms*, and so on. For each value of m , the frequency of the m th harmonic is $m\omega_0$ rad/s, that is, m times the fundamental frequency.

Because the Fourier series is a least-squares approximation, the coefficients in (2.22) and (2.23) must be least-squares coefficients. If we derive one set of these, we can find the other set using Table 2.1, so let us derive the a and b coefficients in (2.23). These may be determined using (2.21), because the harmonic functions are mutually orthogonal with respect to any set of regularly spaced samples over one period of the fundamental function. As in Figure 2.5, we may be thinking of a single sample vector, $x = [x_0, \dots, x_{N-1}]$, or we may be analyzing a periodic function with repetitions of x . In either case,

TABLE 2.1
Equivalence of Fourier Series Coefficients

$[a, b]$ in terms of c	$a_0 = 2c_0$; $a_m = c_m + c_{-m}$; $m > 0$ $b_0 = 0$; $b_m = j(c_m - c_{-m})$; $m > 0$
c in terms of $[a, b]$	$c_0 = a_0/2$; $c_m = (a_m + jb_{-m})/2$; $m < 0$ $c_m = c_{-m}^* = (a_m - jb_{-m})/2$; $m > 0$

if the fundamental period, $2\pi/\omega_0$, covers N samples or NT seconds, then the fundamental frequency must be

$$\omega_0 = \frac{2\pi}{NT} \text{ rad/s} \quad (2.24)$$

With this substitution to indicate sampling over exactly one fundamental period, (2.22) and (2.23) are now expressed as follows:

$$\hat{x}_n \equiv \hat{x}(c, nT) = \sum_{m=-(M-1)}^{M-1} c_m e^{j2\pi mn/N} \quad (2.25)$$

$$= \frac{a_0}{2} + \sum_{m=1}^{M-1} a_m \cos(2\pi mn/N) + b_m \sin(2\pi mn/N) \quad (2.26)$$

In this form, the harmonic functions in (2.26) are orthogonal with respect to the N samples of x . That is,

$$\begin{aligned} \sum_{n=0}^{N-1} \cos(2\pi mn/N) \sin(2\pi kn/N) &= 0; \quad m, k \geq 0 \\ \sum_{n=0}^{N-1} \cos(2\pi mn/N) \cos(2\pi kn/N) &= 0; \quad m, k \geq 0 \text{ and } m \neq k \\ \sum_{n=0}^{N-1} \sin(2\pi mn/N) \sin(2\pi kn/N) &= 0; \quad m, k \geq 0 \text{ and } m \neq k \end{aligned} \quad (2.27)$$

These results may be proved using the trigonometric identities in Table 1.2 and the geometric series application in (1.21).

Because the harmonic functions in (2.23) are all mutually orthogonal, we can apply (2.21) to solve for the least-squares coefficients. In the denominator of (2.21), if g_{mn} is one of the sine or cosine terms in (2.27), we can again use the identities in Table 1.2 for $\sin^2\theta$ and $\cos^2\theta$ and the geometric series formula to show that the denominator of (2.21) is $N/2$. For example,

$$\sum_{n=0}^{N-1} \sin^2(2\pi mn/N) = \sum_{n=0}^{N-1} (1 - 2\cos(2\pi mn/N))/2 = \sum_{n=0}^{N-1} 1/2 - 0 = N/2 \quad (2.28)$$

Thus, using x_n for the sample data in (2.21), the least-squares Fourier coefficients in (2.26) are

$$\begin{aligned} a_m &= \frac{2}{N} \sum_{n=0}^{N-1} x_n \cos(2\pi mn/N); \quad 0 \leq m \leq M-1 \\ b_m &= \frac{2}{N} \sum_{n=0}^{N-1} x_n \sin(2\pi mn/N); \quad 1 \leq m \leq M-1 \end{aligned} \quad (2.29)$$

In using (2.21) to derive (2.29), we note that c in (2.21) stands for any coefficient so long as the orthogonality condition holds. When we are talking about the *Fourier coefficients* in (2.25), the elements of c , which are not all mutually orthogonal, are found using Table 2.1:

$$c_m = \frac{1}{N} \sum_{n=0}^{N-1} x_n e^{-j2\pi mn/N}; \quad m \geq 0 \quad (2.30)$$

$$c_{-m} = c_m^*$$

A summary of the discrete Fourier series in (2.25) and (2.26) and the coefficient formulas we have just derived is presented in Table 2.2. We emphasize again that these formulas work only because the fundamental frequency of the series is $\omega_0 = 2\pi/NT$ rad/s. If we wish to apply the Fourier series to samples where this is not the case, as we did in the example using the samples in (2.14), then we must use the general least-squares solution in (2.13). We now consider an example where the formulas in Table 2.2 can be applied.

Suppose a continuous signal is sampled, and the result is the vector with samples shown on the left in Figure 2.5. The upper equations in Table 2.2 are applied to these samples with $M = 5$ in the following MATLAB file:

```
% Chapter 2: Discrete Fourier Series.
% Index vectors, n and m.
N=50;
n=[0:N-1];
M=5;
m=[0:M-1];
% Sample vector, x.
x=[zeros(1,28),ones(1,12),zeros(1,N-40)];
% a and b coefficients. Note: b0=0 and a0/2 replaces a0.
a=(2/N)*x*cos(2*pi*(n')*m/N);
a(1)=a(1)/2;
b=(2/N)*x*sin(2*pi*(n')*m/N);
xhat=a*cos(2*pi*(m')*n/N)+b*sin(2*pi*(m')*n/N);
```

(2.31)

There are several things to notice in the MATLAB code, especially in the array products. First, the index vectors (row vectors) are set to begin at zero. Then the sample vector x in Figure 2.5 is specified as a row of zeros followed by ones followed by zeros, with a total of N elements. Next, in the computations of a and b , note how the product $(n') * m$ produces an N by M array of sine or cosine values which, when multiplied by x , produce the a and b row vectors. Note that $a(1)$, which is really a_0 , is divided by two to satisfy the form of $\hat{x}_n$ in Table 2.2. Finally, note that $\hat{x}$

TABLE 2.2

Discrete Fourier Series and Coefficient Formulas

$\hat{x}_n = \frac{a_0}{2} + \sum_{m=1}^{M-1} a_m \cos(2\pi mn/N) + b_m \sin(2\pi mn/N)$	$a_m = \frac{2}{N} \sum_{n=0}^{N-1} x_n \cos(2\pi mn/N); \quad 0 \leq m \leq M-1$
	$b_m = \frac{2}{N} \sum_{n=0}^{N-1} x_n \sin(2\pi mn/N); \quad 1 \leq m \leq M-1$
$\hat{x}_n = \sum_{m=-(M-1)}^{M-1} c_m e^{j2\pi mn/N}$	$c_m = \frac{1}{N} \sum_{n=0}^{N-1} x_n e^{-j2\pi mn/N}; \quad 0 \leq m \leq M-1$
	$c_{-m} = c_m^*$

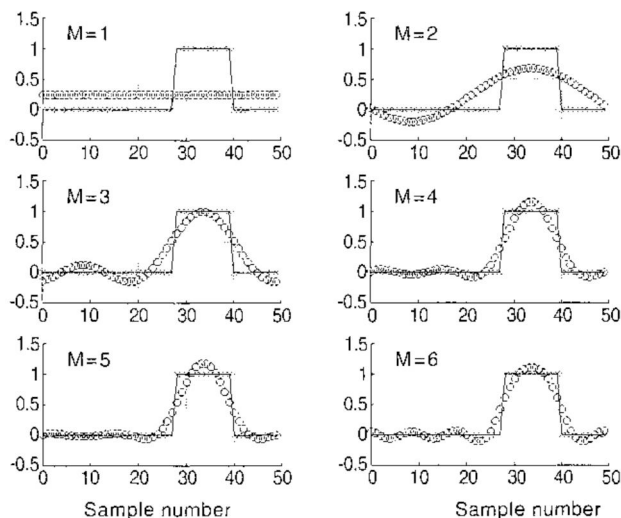


FIGURE 2.6

Discrete Fourier series approximations with increasing values of the series size, M . Waveforms are plotted versus the sample number.

$\hat{x}_n$ in (2.31)] is produced in a similar way, this time using the product $(m') * n$, which produces an M by N array of the indexes. The result, $\hat{x}$, is plotted in Figure 2.6 for increasing values of M to show how the least-squares approximation improves with M .

As with any signal vector x , we may consider the vector elements, x_n and $\hat{x}_n$ in Table 2.2, as samples of a continuous waveform. If we do this, it is reasonable to make two substitutions in Table 2.2 to obtain an approximation, $\hat{x}(t)$, for the continuous waveform. First, we substitute the fundamental frequency, ω_0 in (2.24), and second, we substitute continuous time, t , for nT

as in (2.4). The result is a continuous version of the discrete formulas in Table 2.2:

$$\hat{x}(t) = \frac{a_0}{2} + \sum_{m=1}^{M-1} a_m \cos(m\omega_0 t) + b_m \sin(m\omega_0 t) \quad (2.32)$$

$$\hat{x}(t) = \sum_{m=-(M-1)}^{M-1} c_m e^{jm\omega_0 t} \quad (2.33)$$

Because the coefficients here are derived from samples as in Table 2.2, these results suggest a method for reconstructing a continuous function from a set of samples. We will discuss this idea further in the next chapter.

The results in (2.32) and (2.33) also suggest a continuous form of the Fourier series, that is, a computation of the coefficients such that the reconstruction of $x(t)$ is exact. Instead of a formal development of the continuous form, we offer the following intuitive approach. In the three coefficient formulas in Table 2.2, we allow m to be unlimited and substitute the fundamental frequency, $\omega_0 = 2\pi/NT$ rad/s, so that the formula for c_m , for example, may be written as follows:

$$c_m = \frac{\omega_0}{2\pi} \sum_{n=0}^{N-1} x_n e^{-jm\omega_0 nT} T; \quad 0 \leq m < \infty; \quad c_{-m} = c_m' \quad (2.34)$$

With c_m in this form, we can imagine decreasing the time step, T , toward zero, and at the same time increasing N in a way such that the period, NT , remains constant. The samples (x_n) of $x(t)$ are thus packed more densely, so that, in the limit, we have the Fourier series for a continuous periodic function, $x(t)$:

$$x(t) = \sum_{m=-\infty}^{\infty} c_m e^{jm\omega_0 t}; \quad c_m = \frac{\omega_0}{2\pi} \int_0^{2\pi/\omega_0} x(t) e^{-jm\omega_0 t} dt \quad (2.35)$$

Sometimes, for the sake of symmetry, c_m is given by an integral around $t = 0$ as follows:

$$c_m = \frac{\omega_0}{2\pi} \int_{-\pi/\omega_0}^{\pi/\omega_0} x(t) e^{-jm\omega_0 t} dt \quad (2.36)$$

By separating the integral in (2.35) into two halves and substituting $u = t - 2\pi/\omega_0$ in the second integral, one can see that the two forms of c_m are equivalent. Derivation of formulas for a_m and b_m is left for Exercise 13 at the end of this chapter. In DSP, we are more generally concerned with sampled signals rather than continuous signals. The continuous forms of the Fourier series are, nevertheless, applicable to a wide range of natural periodic phenomena.

In this section, we have introduced two forms of the discrete Fourier series and shown how to calculate the coefficients when the samples are taken over one fundamental period of the data, which is the usual case. In the next chapters, we will see that harmonic analysis, and hence the Fourier series, forms the basis for the discrete Fourier transform and for spectral analysis in general.

2.5 Exercises

1. Find the parabola, that is, Equation (2.3) with $M = 3$, that forms a least-squares fit to each set of data points given in (a) through (d) below. In each case, assume that the time step, T , is one, so the samples in f are taken at $t = 0, 1, \dots, N - 1$, where N is the length of f . Write a MATLAB file to perform the following steps:
 - Compute the vectors G_1 , G_2 , and G_3 in (2.9); that is, the columns of G in (2.11).
 - Form the array G in (2.11).
 - Solve for the least-squares coefficients, c_1 , c_2 , and c_3 , using (2.13).
 - Make a continuous plot of the least-squares parabola, $\hat{f}(t)$, in the range $t = [0, N - 1]$. Use 1000 values to produce a smooth plot.
 - Finally, on the same graph, plot the elements of f as discrete points. Label the plot using functions *xlabel*, *ylabel*, and *title* and observe whether the least-squares fit seems to be correct.
 - a. $f = [1 \ 3 \ 2]$
 - b. $f = [1 \ 3 \ 4 \ 2]$
 - c. $f = [1.1 \ 2.5 \ 3.2 \ 3.8 \ 3.7 \ 3.1 \ 2.0]$
 - d. $f = [8 \ 6 \ 4 \ 2.5 \ 1 \ 0 \ -0.5 \ -0.5 \ -0.5 \ 0 \ 1.5 \ 2.5 \ 4 \ 5 \ 7 \ 8]$
2. Try to execute the program in Exercise 1 using a vector $f = [-1 \ 3]$, that is, a vector with $N < M$.
 - a. Print the coefficient matrix and the right-hand vector in (2.10).
 - b. Show from (a) why the three linear equations are not independent, and why therefore, a unique solution for c does not exist.
 - c. Show what happens when the MATLAB backslash, (matrix)\(vector), is attempted.
 - d. Explain in simple geometric terms why the three equations do not have a single unique solution.

3. Some dates and closing values of the Dow-Jones Industrial Average during 1960 were as follows:

Yr	Mo	Da	Close	Yr	Mo	Da	Close	Yr	Mo	Da	Close
1960	1	4	679.060	1960	5	3	607.730	1960	8	31	625.990
1960	1	22	645.850	1960	5	23	623.660	1960	9	21	594.260
1960	2	11	618.570	1960	6	13	655.850	1960	10	11	588.750
1960	3	3	612.050	1960	7	1	641.300	1960	10	31	580.360
1960	3	23	622.060	1960	7	22	609.870	1960	11	21	604.540
1960	4	12	626.500	1960	8	11	622.880	1960	12	12	611.940

- Using steps similar to those in Exercise 1, find and print the coefficients of a least-squares straight line ("trend") using this data. (Hint: change the dates to fractions of a year using a reasonable approximation.)
- Plot the data points and the trend line on the same graph. Make the ordinate range $[0, 1000]$ and extend the trend line beyond 1960 through the first half of 1961.
- Also (on the same graph) plot the following closing values. Was the 1960 trend valid in the first half of 1961?

Yr	Mo	Da	Close	Yr	Mo	Da	Close
1961	1	3	610.250	1961	4	13	692.020
1961	1	23	639.820	1961	5	3	688.900
1961	2	10	639.670	1961	5	23	700.590
1961	3	3	671.570	1961	6	14	695.810
1961	3	23	675.450				

4. The intensity, I , of a certain extraterrestrial source is known to vary sinusoidally around a constant value, c_0 , with a period of 1000 years. We have the following history:

Date	Intensity (I)	Date	Intensity (I)	Date	Intensity (I)
1/1/1900	167	1/1/1940	105	1/1/1980	42
1/1/1910	152	1/1/1950	89	1/1/1990	27
1/1/1920	136	1/1/1960	74	1/1/2000	11
1/1/1930	121	1/1/1970	58		

- If time (t) is measured in years, what is the fundamental frequency (ω_0) of I in rad/year?
- Using steps similar to those in Exercise 1, find, c_0 , c_1 , and c_2 , so that the function

$$\hat{f}(t) = c_0 + c_1 \sin \omega_0 t + c_2 \cos \omega_0 t$$

is a least-squares fit to the data.

- Plot $\hat{f}(t)$ from year 1900 through year 2999, and plot the sample values on the same graph.

- d. Explain why the Fourier coefficient formulas in Table 2.2 are not valid in this exercise.
5. Suppose we have two sample vectors, x and y , each with $N + 1$ samples taken at $t = 0, T, 2T, \dots, NT$ from the functions $x(t) = a(t - NT/2)$ and $y(t) = x^2(t)$, where a and b are constants.
 - a. Write a program to show whether or not x and y are orthogonal for $N = 100$ and $N = 101$. For convenience, use $a = b = T = 1$.
 - b. Prove that x and y are orthogonal for any $N > 0$.
6. Prove that two different harmonic functions, $e^{-j2\pi mn/N}$ and $e^{-j2\pi mn/N}$, in the complex Fourier series are orthogonal with respect to the set of samples at $n = 0, 1, \dots, N - 1$.
7. Two theorems are important and helpful when applying the Fourier series to a sampled periodic function, say $x(t)$, when the samples are at $t = 0, T, \dots, (N - 1)T$. The first theorem is

If $x(t)$ is an even function, that is, $x(nT) = x(-nT) = x((N - n)T)$, then the b_m coefficients are all zero, $c_m = c_{-m}$ is real, and the Fourier series is a cosine series.

- a. Assume N is even. In the formula for b_m in Table 2.2, show that the terms in the sum for $n = 0$ and $n = N/2$ are both zero.
 - b. Show that if $x(t)$ is even, $x_n = x_{N-n}$ for any value of n .
 - c. Using (b), separate the nonzero terms in the sum into two sums that cancel, thus proving the theorem.
8. The second theorem is similar to the first theorem in Exercise 7:

If $x(t)$ is an odd function, that is, $x(nT) = -x(-nT) = -x((N - n)T)$, then the a_m coefficients are all zero, $c_m = -c_{-m}$ is imaginary, and the Fourier series is a sine series.

- a. Assume N is even. In the formula for a_m in Table 2.2, show that, since $x_n = x_{-n}$, the terms in the sum for $n = 0$ and $n = N/2$ must both be zero.
 - b. Show that if $x(t)$ is odd, $x_n = -x_{N-n}$ for any value of n .
 - c. Using (b), separate the nonzero terms in the sum into two sums that cancel, thus proving the theorem.
9. The triangular waveform in Figure 2.1 is seen to have amplitude 5.0 and period 500.
- a. Write a MATLAB expression to construct a row vector, x , of samples of the waveform $x(t)$ over exactly one period beginning at $t = 0$ and using sampling interval $T = 0.5$.
 - b. Which of the theorems in Exercises 7 and 8 applies to this waveform?

- c. Using the appropriate theorem, find and print the Fourier series coefficients for the first form of the series in Table 2.2 for $M = 6$.
- d. Using the first form of the series in Table 2.2, plot exactly one cycle of the original waveform, $x(t)$, and the Fourier series approximation, $\hat{x}_n$ with $M = 6$, versus t on the same graph.
- 10. Repeat Exercise 9, but this time use $T = 25$ and do part (d) differently. Instead of plotting the original waveform, plot x as a set of discrete points. Then plot the continuous approximation, $\hat{x}(t)$ in (2.32), using 2000 samples of t .
- 11. A periodic binary (two-valued) waveform has a fundamental period equal to $1\ \mu\text{s}$ (microsecond). One hundred samples are taken at $t = 0.00, 0.01, \dots, 0.99\ \mu\text{s}$. The sample vector x consists of 20 ones followed by 61 zeros followed by 19 ones.
 - a. Do either of the theorems in Exercises 7 and 8 apply to this vector? If so, which?
 - b. Using $M = 16$, compute the complex Fourier series coefficients, c_m in Table 2.2, for $m = 0, 1, \dots, M - 1$. Do the computed values agree with your answer to (a)?
 - c. Plot x as a set of discrete points, and plot the continuous approximation, $\hat{x}(t)$ in (2.32), using 1000 samples of t .
- 12. The average monthly high and low temperatures ($^{\circ}\text{F}$) in Albuquerque, NM, are as follows:

Month:	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Avg. High:	46	53	61	70	79	90	92	89	81	71	57	47
Avg. Low:	21	26	32	39	48	58	64	62	55	43	31	23

- a. Do either of the theorems in Exercises 7 and 8 apply to this vector? If so, which?
- b. Compute the Fourier series coefficients a_0 , a_1 , and b_1 in Table 2.2.
- c. Plot each temperature vector as a set of discrete points, and plot each continuous approximation using (2.32) with 1000 samples of t . Make all plots on the same graph with the abscissa labeled in months, that is, January = 1, February = 2, etc.
- d. Comment on why the least-squares fit is as good as it is with $M = 2$. In other words, why is the average annual temperature variation nearly sinusoidal?
- 13. Complete the following:
 - a. Derive formulas similar to those in (2.35) for the continuous Fourier series in terms of sine and cosine functions. (Hint: Use the formula involving $e^{j\theta}$ in Chapter 1, Table 1.2.)
 - b. Give formulas similar to (2.36) for the Fourier series coefficients found in part (a).

References

Least Squares

1. Giordano, A.A. and Hsu, F.M., *Least Square Estimation with Applications to Digital Signal Processing*, John Wiley & Sons, New York, 1985.

Fourier Series

2. Bellanger, M., *Digital Processing of Signals*, 2nd ed., John Wiley & Sons, New York, 1989, chap. 1.
3. Lyons, L., *All You Wanted to Know About Mathematics But Were Afraid to Ask*, Cambridge Press, Cambridge, U.K., 1998, chap. 12.

Correlation, Fourier Spectra, and the Sampling Theorem

3.1 Introduction

The word *spectrum* has several meanings. In signal processing, the spectrum of a signal is a particular kind of mapping function. In the case of a waveform, the spectrum maps the signal from the *time domain* into the *frequency domain*, where we can see exactly how the signal content is distributed over frequency. In the case of an image, the spectrum maps the signal from the *spatial domain* of the image into a two-dimensional frequency domain. Sometimes the spectrum of a waveform or an image shows us things we can only see less clearly or even not see at all, when we look at the signal in the time domain. We will see some examples of this later in this chapter.

In this chapter, we discuss the *Fourier spectrum* of a signal in terms of the *discrete Fourier transform* and the *continuous Fourier transform*. (Later, we will discuss other transforms as well.) These two transforms allow us to obtain the spectral properties of continuous and discrete (sampled) signals.

The two Fourier transforms also provide a means to prove the *Sampling Theorem*, which is basic to all DSP. The sampling theorem tells us how often a given continuous signal must be sampled in order to preserve all of the information in the signal, so that we are able to compute the spectrum correctly and even reconstruct the continuous signal from the samples. The sampling theorem then allows us to discuss *aliasing*, which is something that happens to the spectrum when a signal is not sampled often enough.

The final subject in this chapter is *waveform reconstruction*, that is, methods for obtaining or estimating the original continuous waveform from a set of discrete samples.

Before we begin with these subjects, we introduce another operation known as *correlation*. Correlation has many uses in signal processing, but it is introduced here for one main reason: the discrete Fourier spectrum of a signal, $x(t)$, can be understood as a correlation of $x(t)$ with a set of sine and cosine functions.

3.2 Correlation

We begin the discussion of spectra by reviewing *correlation* as it applies to signal processing. Basically, correlation is a measure we use to tell how much two waveforms or two images are like each other. We also use *autocorrelation* to tell how much a waveform at time t is like itself at some other time, $t + \tau$, or how much an image at location (x, y) is like itself at some other location, $(x + \sigma, y + \tau)$.

Suppose we have two waveforms, $x(t)$ and $y(t)$, defined in the range $t = [0, \infty)$. (This notation means the signals are defined from the lower limit to just before the upper limit.) Then, the correlation function of x and y is

$$\phi_{xy}(\tau) = E\{x(t)y(t+\tau)\} = \lim_{L \rightarrow \infty} \frac{1}{L} \int_0^L x(t)y(t+\tau) dt; \quad \tau \geq 0 \quad (3.1)$$

where $E[\bullet]$ stands for the *expected value*. The independent variable τ has the effect of shifting y forward in time relative to x , thus making the correlation function a continuous function. For sampled waveforms, the correlation function is the discrete form of (3.1), that is,

$$\phi_{xy}(m) = E\{x_n y_{n+m}\} = \lim_{L \rightarrow \infty} \frac{1}{L} \sum_{n=0}^{L-1} x_n y_{n+m}; \quad m \geq 0 \quad (3.2)$$

In this case, the time shift, m , is measured in samples.

We can observe in either (3.1) or (3.2) that the correlation function, as defined here, is the *average product* of two signals. The product is averaged without an upper bound so that, as long as the time shift is positive, we do not need to be concerned with end effects. If the waveforms are defined only in the finite interval $[0, NT)$, then we usually make one of two assumptions:

The waveform values are zero outside the interval.

The waveform is periodic with period NT , as illustrated on the right in Figure 2.5.

An example of the correlation of two waveforms is given in Figure 3.1, in which $x(t)$ is the rectangular signal, and $y(t)$ is the triangular signal. Both signals are defined in the expressions for vectors x and y in the MATLAB code (3.3). The MATLAB code computes the correlation function $\phi_{xy}(m)$ using (3.2) with periodic extension, recognizing that the average product in (3.2) can be computed by averaging over one period. Note how the use of yy , the one-period extension of y , is used to prevent y_{n+m} in (3.2) from going beyond the end of the vector.